

Personal Project Related Work &
Technical Background
Minor AI for Society

Casey Stijlaart

Table of Contents

1. Document History	1
2. Introduction.....	1
3. Related Work.....	2
3.1. IoT-Based Plant Monitoring Systems	2
3.2. AI/ML-Based Plant Health Monitoring Systems.....	3
3.3. Commercial Solutions	4
3.4. Comparison of Existing Systems	5
3.5. Summary of Existing Plant Monitoring Systems.....	5
4. Technical Background.....	6
4.1. Embedded Systems for Edge AI.....	6
4.2. Sensor Data Analysis	8
4.3. Datasets.....	10
4.4. Model Overview	12
4.5. Validation Strategy.....	14
4.6. Summary of Technical Background.....	15
5. References	16

1. Document History

Table 1. Document History

Version	Date	Status	Author	Description
0.1	18-03-2026	Initial Version	Casey Stijlaart	Related Work
0.2	18-03-2026	Initial Version	Casey Stijlaart	Technical Background; Edge AI, Sensor Data
1.0	20-03-2026	First Version	Casey Stijlaart	Completed First Version

2. Introduction

This document serves as a research report for the related work and technical background of the personal project for the Minor AI for Society course. The purpose of this report is to provide an overview of the related work in the field of AI and its applications in society, as well as the technical background necessary for the development of the project. The report is structured into two main sections: Related Work and Technical Background. The Related Work section will review existing research and applications of AI in society, while the Technical Background section will cover the fundamental concepts and technologies that underpin the project.

3. Related Work

This section reviews existing research and commercial solutions related to plant health monitoring systems, with a focus on IoT-based approaches, AI/ML techniques, and the integration of these technologies in practical applications. The review is structured into three main categories: IoT-Based Plant Monitoring Systems, AI/ML-Based Plant Health Monitoring Systems, and Commercial Solutions. Each category highlights key studies, methodologies, and findings that inform the design and development of the proposed system. The section concludes with a summary of the existing systems and their relevance to the project.

3.1. IoT-Based Plant Monitoring Systems

Currently, a variety of plant monitoring systems are available, most of which have been developed by students, hobbyists, or small-scale developers. These systems generally focus on environmental monitoring rather than advanced AI-based analysis.

Many of these solutions are open-source, providing access to hardware and software designs, while others are proprietary. The majority of existing systems do not integrate AI or Machine Learning; instead, they monitor environmental parameters—such as soil moisture, temperature, and humidity—and notify users when predefined thresholds are exceeded.

Some implementations are more advanced, incorporating automated irrigation systems that water plants when certain conditions are met, improving plant care efficiency. Other systems are simpler, only sending alerts or notifications to the user without any automation.

Examples of existing systems include:

- Plant Monitoring System by Shishir3D
- IoT-based Plant Health Care System (MDPI)
- IoT-Based Plant Health Care System (Academia.edu)
- IoT-Based Smart Irrigation System (Academia.edu)

3.2. AI/ML-Based Plant Health Monitoring Systems

In contrast to traditional IoT-based plant monitoring systems, recent developments have begun integrating Artificial Intelligence (AI) and Machine Learning (ML) techniques. These systems are capable of not only monitoring environmental parameters but also analyzing data to make intelligent predictions, detect anomalies, and optimize plant care.

AI/ML-based plant health monitoring systems can:

- Predict irrigation needs based on soil moisture trends and weather conditions.
- Detect early signs of plant stress or disease using sensor data or images.
- Provide automated recommendations or interventions to improve plant health.
- Learn and adapt to the specific requirements of different plant species over time.

These advancements enable more proactive and efficient plant management, surpassing the reactive threshold-based alerts of traditional systems.

Examples of AI/ML-based systems include:

- IoT-Based Smart Plant Monitoring System (Emergent Mind) []
- AI-Enabled IoT-Based Smart Indoor Plant Care System (Academia.edu) []
- Deep Learning for Plant Health Monitoring (Nature Scientific Reports) []
- AI-Driven Smart Plant Monitoring (IJSAT Journal) []
- Plant Health Monitoring System (GitHub) []

These systems illustrate the potential for combining IoT hardware with AI/ML algorithms to achieve intelligent, autonomous plant care and monitoring.

These existing approaches highlight the potential for enhancement by integrating AI and ML techniques, which could enable predictive care, anomaly detection, and intelligent decision-making for plant health and irrigation management.

3.3. Commercial Solutions

Several commercial solutions exist for crop and plant monitoring; however, most of these systems are designed for large-scale agricultural applications rather than small-scale or individual plant care. These solutions often integrate advanced sensors and IoT platforms to monitor soil conditions, plant health, and environmental factors at scale.

While they provide reliable data and automated management for farms and commercial crops, their focus and cost make them less suitable for typical household or small-scale plant monitoring projects. Nonetheless, these commercial systems demonstrate the potential of IoT and sensor integration for predictive and automated plant care.

Examples of commercial systems include:

- Connected Green – Soil Moisture Sensor []: A commercial sensor solution for monitoring soil moisture in large crop environments. Available at: <https://connectedgreen.nl/bodemvochtsensor>
- Cropwise Interra Scan []: A precision agriculture platform that uses IoT devices and analytics for crop monitoring and management. Available at: <https://www.cropwise.com/netherlands/interra-scan>
- Vivent Biosignals – Food and Agriculture Inputs []: Provides sensor-based monitoring and analytics for plant and crop health on a commercial scale. Available at: <https://vivent-biosignals.com/solutions/food-and-ag-inputs>

These systems illustrate how IoT and sensor technologies are applied in commercial agriculture, but also highlight the gap for small-scale, AI/ML-based plant care systems suitable for personal or hobbyist use.

3.4. Comparison of Existing Systems

Table 2. Comparison Table of Plant Monitoring Systems

Feature / System Type	IoT-Based (Non-AI)	AI/ML-Based	Commercial Solutions
Typical Cost	Low (€5–50)	Low–Medium (Research prototypes €20–100)	High (€100+ per sensor / system)
Target Use	Hobbyist / student projects	Research / experimental	Commercial crops / large-scale agriculture
Automation	Some (watering, notifications)	Advanced (predictive watering, alerts, recommendations)	Advanced (field irrigation, crop management)
AI/ML Integration	No	Yes (predictive and adaptive)	Limited (mostly analytics for large-scale optimization)
Sensors Used	Basic environmental (soil moisture, temperature, humidity)	Environmental + image/spectral data	Advanced multi-sensor arrays, remote sensing
Open Source Availability	Often open source	Mostly research / prototype	Rarely open source
Scalability	Easy for small-scale deployment	Moderate, experimental	High for large fields, not small-scale
Gaps Identified	Lacks AI/ML, predictive capabilities	Limited accessibility and cost-effective deployment	Not suitable for indoor/home plants, high cost

3.5. Summary of Existing Plant Monitoring Systems

Plant monitoring systems can be grouped into three categories:

- **IoT-Based (Non-AI/ML):** Monitor parameters like soil moisture, temperature, and humidity, often triggering alerts or basic automation Widely used in hobbyist projects, but lack predictive intelligence
- **AI/ML-Based Systems:** Research prototypes that predict irrigation needs, detect stress or disease, and provide adaptive recommendations Mostly experimental with limited consumer adoption
- **Commercial Solutions:** Large-scale agricultural systems with advanced sensors and IoT platforms Highly capable but costly and unsuitable for indoor or small-scale plants

Overall, there is a gap for **affordable, small-scale AI/ML-based plant monitoring**, combining intelligent predictions with IoT for personal plant care.

4. Technical Background

This section provides an in-depth technical background relevant to the design and implementation of the plant health monitoring system. It covers various AI/ML approaches, datasets, model research, embedded system constraints, validation strategies, and design implications that inform the development of the project.

4.1. Embedded Systems for Edge AI

4.1.1. Low-Power Microcontrollers

Low-power microcontrollers enable embedded artificial intelligence systems by combining compact size with energy-efficient computation. These devices can interface directly with environmental sensors while performing real-time inference locally, which is critical for applications such as autonomous plant monitoring. Research and community projects have demonstrated that even constrained architectures like the ESP32 family can execute lightweight machine learning workloads when models are optimized for on-device inference.

4.1.2. TinyML Concept

Tiny Machine Learning (TinyML) refers to deploying optimized machine learning models on microcontrollers and other resource-constrained embedded devices. TinyML workflows focus on converting trained models (e.g., small neural networks or classical classifiers) into lightweight, quantized representations that can run efficiently with limited RAM and processing power.

Key aspects of TinyML include:

- **Quantization:** Reducing model weights and activations to 8-bit or lower precision.
- **Memory efficiency:** Ensuring models fit within small flash and SRAM capacities.
- **Optimization:** Tailoring model architecture for real-time inference on low-power hardware.

In plant monitoring systems, TinyML enables:

- Real-time event detection such as soil moisture thresholds or plant stress.
- Minimal dependency on cloud infrastructure, reducing latency and preserving privacy.
- Energy-efficient operation, allowing prolonged deployment in remote or battery-powered environments.

4.1.3. Edge AI vs Hybrid AI

Edge AI and hybrid AI represent different approaches for deploying machine learning in embedded systems:

- **Edge AI:** All inference occurs locally on the device, with data captured, processed, and classified in real-time. Advantages include low latency, independence from network connectivity, and improved privacy. This is particularly suitable for plant monitoring systems operating in greenhouses or outdoor areas where connectivity is limited.
- **Hybrid AI:** Combines local edge inference with cloud-based processing. Lightweight models handle immediate tasks on-device, while complex analytics or model retraining occurs in the cloud. This approach allows more sophisticated analysis but relies on reliable connectivity and may introduce latency.

For autonomous plant monitoring, Edge AI is generally preferred for resilience, energy efficiency, and responsiveness, while hybrid AI may support periodic model updates or long-term analytics.

4.1.4. Edge AI on ESP32

Although documentation specific to the ESP32 CH2102 is limited, several resources demonstrate the feasibility of Edge AI on ESP32-class microcontrollers:

- **Edge Impulse on ESP32:** Provides end-to-end TinyML workflows including data acquisition, training, and deployment to ESP32 boards for tasks such as vision and sensor-based classification.
- **Community projects:** Open-source projects show TensorFlow Lite Micro and Edge Impulse models running on ESP32 devices for plant health monitoring, demonstrating real-time on-device inference and sensor integration.
- **Research applications:** ESP32 microcontrollers have been used in IoT-based smart irrigation and plant monitoring systems where lightweight ML models predict watering needs or detect environmental stress.

These examples collectively indicate that Edge AI is practical and effective on ESP32-class MCUs, including variants like the CH2102.

4.1.5. Edge AI Workflow

The workflow of how Edge AI operates in practice is shown in the diagram below. It illustrates the process from sensor data acquisition to on-device inference and actuator/alert response.

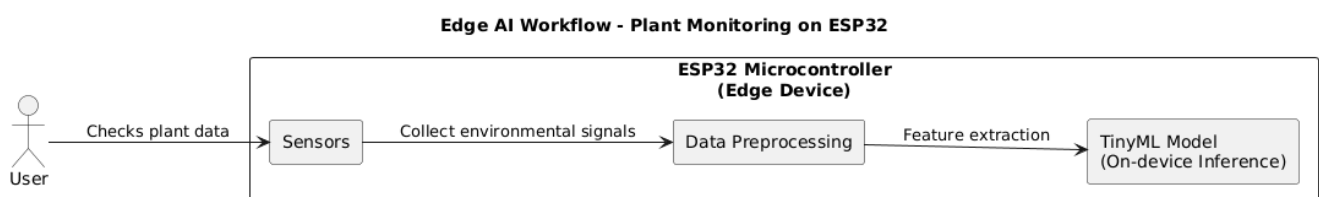


Figure 1. Edge AI Workflow

4.2. Sensor Data Analysis

In a plant monitoring system, sensor data is the foundation of all decision-making. Efficient analysis of this data on low-power microcontrollers requires careful consideration of sensor types, preprocessing techniques, and anomaly detection strategies.

4.2.1. Types of Sensor Data

Plant monitoring systems typically rely on multiple environmental and plant-specific sensors, such as:

- **Soil Moisture Sensors:** Measure volumetric water content to determine irrigation needs.
- **Temperature and Humidity Sensors:** Track ambient conditions that affect plant growth.
- **Light Sensors (e.g., PAR or LDR):** Assess sunlight exposure and photosynthetic potential.
- **Additional sensors:** CO₂ concentration, pH, nutrient levels, or plant stress indicators can be added depending on system requirements.

Each sensor produces data streams that are generally time-series in nature, making temporal patterns important for detecting anomalies or triggering automated responses.

4.2.2. Time-Series Anomaly Detection

Time-series anomaly detection is critical in plant monitoring for identifying abnormal conditions such as drought stress, overwatering, or environmental irregularities. On low-power devices like the ESP32, lightweight algorithms are preferred:

- **Statistical Methods:** Z-score, moving average deviations, or seasonal trend decomposition.
- **Machine Learning Methods:** Small neural networks or auto encoders trained to detect deviations from normal patterns.
- **Threshold-Based Alerts:** Simple, rule-based triggers suitable for on-device real-time monitoring.

By detecting anomalies locally, the system can act accordingly without relying on cloud connectivity.

4.2.3. Lightweight Preprocessing

Preprocessing sensor data before model inference is essential on constrained devices to reduce memory and computation requirements. Techniques include:

- **Normalization and Scaling:** Adjusting sensor values to a standard range suitable for ML models.
- **Noise Filtering:** Using low-pass filters or moving averages to remove transient spikes.
- **Feature Extraction:** Calculating simple features such as rolling means, variance, or delta changes to summarize sensor signals.
- **Dimensionality Reduction:** Combining correlated sensor readings or reducing sampling frequency to decrease data size.

Efficient preprocessing ensures that the TinyML model receives clean, compact, and informative input, enabling accurate predictions while conserving ESP32 resources.

4.3. Datasets

Datasets are essential for developing and evaluating AI/ML models in plant monitoring systems. They enable models to learn patterns in environmental conditions, detect anomalies, and classify plant health states. For this project, datasets that include explicit plant health labels are particularly valuable, as they enable supervised learning approaches.

While traditional research datasets focus on large-scale agriculture, platforms such as Kaggle provide a growing number of accessible, labeled datasets suitable for experimentation and prototyping.

4.3.1. Public Sensor Datasets

Several datasets provide time-series sensor data for plant monitoring:

- **Soil Moisture and Environmental Datasets:** Datasets containing soil moisture, temperature, humidity, and light measurements support time-series analysis and anomaly detection tasks.
- **IoT-based Plant Monitoring Datasets:** These datasets combine multiple environmental sensors and are often used for irrigation prediction and plant condition monitoring.
- **Kaggle Sensor + Health Datasets:** Recent datasets available on Kaggle combine sensor data with plant health status labels, enabling supervised learning:
 - Smart plant biosensor monitoring datasets
 - Indoor plant growth and health datasets
 - Structured plant health datasets with labeled conditions

These datasets are particularly useful for training models that map sensor inputs to plant health classification, which aligns directly with this project.

4.3.2. Image-Based Health Datasets

In addition to sensor data, image-based datasets are widely used for plant health classification. For example, Kaggle offers a large large collections of labeled plant leaf images allow training of classification models for detecting diseases and health conditions.

- These datasets typically include:
 - Healthy vs diseased classifications
 - Multiple disease categories
 - Annotated images suitable for deep learning models

Large-scale datasets can contain tens of thousands of labeled images across multiple disease classes, making them suitable for training robust computer vision models.

Although image-based models are more computationally demanding, they can be combined with sensor-based approaches in hybrid systems.

4.3.3. Data Collection Considerations

Despite the availability of public datasets, custom data collection remains important. Key considerations include:

- **Label availability:** Datasets with explicit health labels (e.g., “healthy”, “stressed”, “diseased”) are essential for supervised learning.
- **Sampling consistency:** Time-series data must be collected at regular intervals to preserve temporal relationships.
- **Sensor reliability:** Calibration and noise reduction are necessary to ensure data quality.
- **Context specificity:** Indoor plant environments differ significantly from agricultural datasets, requiring domain-specific data.
- **Edge constraints:** Data should be lightweight and efficient to process on embedded systems.

4.3.4. Limitations

Despite the increasing availability of datasets, several limitations remain:

- **Mismatch with use case:** Many datasets focus on large-scale agriculture rather than small-scale or indoor plant monitoring.
- **Limited multimodal datasets:** Few datasets combine both sensor data and plant health labels, which are needed for robust Edge AI systems.
- **Data quality issues:** Public datasets (especially from platforms like Kaggle) may contain noise, missing values, or inconsistencies.
- **Computational constraints:** Image-based datasets are often too large for direct deployment on microcontrollers without significant optimization.
- **Domain gap:** Models trained on agricultural datasets may not generalize well to home or indoor plant environments.

As a result, an effective approach is to first test with the existing datasets from e.g. Kaggle. Based upon the results of this test, it can be concluded whether the datasets can directly be used, or if they should be combined with a custom-collected dataset.

4.4. Model Overview

Selecting an appropriate machine learning model is critical for deploying AI on embedded systems. In the context of plant monitoring, models must balance **accuracy, interpretability, and computational efficiency**. Due to the constraints of microcontrollers, lightweight models are preferred, often requiring optimization techniques such as quantization and model pruning.

4.4.1. Lightweight ML Models

Lightweight machine learning models are specifically designed to operate under limited memory and processing resources. For embedded systems, these models must fit within small memory footprints while maintaining acceptable performance.

Common lightweight models include:

- **Linear Models (e.g., Linear Regression):** Suitable for predicting continuous values such as soil moisture trends or temperature changes. These models are simple, fast, and require minimal resources.
- **Decision Trees:** Effective for classification tasks such as determining plant health states (e.g., “healthy”, “dry”, “overwatered”). They are interpretable and computationally inexpensive.
- **k-Nearest Neighbors (k-NN):** Can be used for classification based on similarity between sensor readings. However, it may be less efficient due to memory requirements for storing training data.
- **Small Neural Networks:** Shallow neural networks with few layers can be used for more complex pattern recognition, such as combining multiple sensor inputs. These models are often deployed using TensorFlow Lite for Microcontrollers.
- **Anomaly Detection Models:** Lightweight approaches such as thresholding, moving averages, or simple autoencoders can detect abnormal plant conditions in time-series data.

These models are commonly used in TinyML applications due to their **low computational cost and adaptability**.

4.4.2. Time-Series Prediction Basics

Sensor data in plant monitoring systems is inherently **time-series data**, meaning that observations are collected sequentially over time. This allows models to learn temporal patterns and predict future states.

Key concepts include:

- **Trend:** Long-term changes in sensor values (e.g., gradual soil drying).
- **Seasonality:** Repeating patterns over time, such as daily light cycles.
- **Noise:** Random fluctuations caused by sensor inaccuracies or environmental disturbances.

Basic approaches for time-series prediction include:

- **Moving Average Models:** Smooth data to identify trends and reduce noise.

- **Autoregressive Models:** Use previous values to predict future values.
- **Sliding Window Methods:** Use a fixed number of past observations as input features for machine learning models.
- **Lightweight Neural Networks:** Simple feedforward networks can approximate temporal relationships when combined with windowed input data.

On embedded systems, simpler methods are often preferred due to their efficiency and ease of implementation.

4.4.3. Suitability for ESP32 / TinyML

When deploying models on microcontrollers such as the ESP32, several constraints must be considered:

- **Memory limitations:** Models must fit within limited RAM and flash memory, often requiring quantization (e.g., 8-bit models).
- **Processing power:** Complex models with high computational requirements are not feasible for real-time inference.
- **Energy consumption:** Efficient models are necessary to maintain low power usage in continuous monitoring systems.
- **Latency requirements:** Real-time responses (e.g., triggering irrigation) require fast inference times.

Based on these constraints, the most suitable models for ESP32-based plant monitoring systems are:

- Decision trees or rule-based models for classification tasks
- Linear regression for simple predictions
- Small neural networks deployed using TensorFlow Lite for Microcontrollers
- Threshold-based or statistical anomaly detection methods

More complex models, such as deep convolutional neural networks, are generally unsuitable for direct deployment on microcontrollers without significant optimization or offloading to external systems.

Therefore, the model selection process must prioritize **efficiency and reliability**, ensuring that the system can perform real-time inference while operating within the constraints of embedded hardware.

4.5. Validation Strategy

This section outlines the planned approach for evaluating the performance of the proposed AI/ML models. As implementation and data collection are still in early stages, this section focuses on the methodology that will be used in later development phases.

4.5.1. Basic Dataset Splitting

To evaluate model performance, datasets will be divided into:

- **Training set:** Used to train the model.
- **Validation set:** Used to tune model parameters.
- **Test set:** Used for final evaluation.

A common split such as 70/15/15 or 80/20 will be used, depending on dataset size. For time-series data, care will be taken to preserve temporal order to avoid data leakage.

4.5.2. Evaluation Metrics

Model performance will be evaluated using appropriate metrics depending on the task:

Classification tasks (plant health):

- Accuracy
- Precision and recall
- F1-score

Regression tasks (e.g., soil moisture prediction):

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

Anomaly detection:

- Detection rate
- False positive rate

These metrics provide insight into both model correctness and reliability.

4.5.3. Performance Considerations

In addition to model accuracy, performance on embedded hardware will be evaluated based on:

- **Inference time:** Ensuring real-time responsiveness on the microcontroller.
- **Memory usage:** Model size must fit within available RAM and flash memory.
- **Energy consumption:** Efficient models are required for continuous operation.
- **Robustness:** The model should perform reliably under noisy or incomplete sensor data.

These constraints are critical for deployment on resource-constrained devices such as the ESP32.

4.6. Summary of Technical Background

This technical background outlines the foundations for an AI-driven plant monitoring system on embedded hardware. Low-power microcontrollers combined with TinyML enable real-time, on-device inference using sensor data such as soil moisture, temperature, and light.

Sensor data is inherently time-series, requiring lightweight preprocessing and analysis techniques to extract meaningful patterns. Public datasets, including those from platforms such as Kaggle, support model development, although custom data collection remains important for domain-specific accuracy.

Lightweight models such as decision trees, linear models, and small neural networks are most suitable for deployment on constrained devices. A validation strategy has been defined to evaluate model performance based on accuracy, efficiency, and hardware constraints.

Overall, this foundation supports the development of an efficient, autonomous Edge AI system for plant monitoring on devices such as the ESP32.

5. References

- [plantmonitoring_github] Shishir3D. **Plant Monitoring System**. Available at: <https://github.com/Shishir3D/PlantMonitoringSystem>
- [plantmonitoring_mdpi] MDPI. **IoT-Based Plant Monitoring System**. Available at: <https://www.mdpi.com/2673-4591/32/1/12>
- [planthealth_academia] Academia.edu. **IoT-Based Plant Health Care System**. Available at: https://www.academia.edu/127680898/IOT_BASED_PLANT_HEALTH_CARE_SYSTEM
- [smartirrigation_academia] Academia.edu. **IoT-Based Smart Irrigation System**. Available at: https://www.academia.edu/117576198/IoT_Based_Smart_Irrigation_System
- [ai_plant_emergentmind] Emergent Mind. **IoT-Based Smart Plant Monitoring System**. Available at: <https://www.emergentmind.com/topics/iot-based-smart-plant-monitoring-system>
- [ai_plant_academia] Academia.edu. **AI-Enabled IoT-Based Smart Indoor Plant Care System**. Available at: https://www.academia.edu/145072019/AI_ENABLED_IOT_BASED_SMART_INDOOR_PLANT_CARE_SYSTEM
- [ai_plant_nature] Nature Scientific Reports. **Deep Learning for Plant Health Monitoring**. Available at: <https://www.nature.com/articles/s41598-025-34681-1>
- [ai_plant_ijssat] IJSAT Journal. **AI-Driven Smart Plant Monitoring**. Available at: <https://www.ijssat.org/papers/2025/1/2971.pdf>
- [ai_plant_github] CodeByPinar. **Plant Health Monitoring System**. Available at: <https://github.com/CodeByPinar/Plant-Health-Monitoring>
- [commercial_connectedgreen] Connected Green. **Soil Moisture Sensor**. Available at: <https://connectedgreen.nl/bodemvochtsensor>
- [commercial_cropwise] Cropwise. **Interra Scan Platform**. Available at: <https://www.cropwise.com/netherlands/interra-scan>
- [commercial_vivent] Vivent Biosignals. **Food and Agriculture Inputs**. Available at: <https://vivent-biosignals.com/solutions/food-and-ag-inputs>
- [esp32_edgeimpulse] Edge Impulse. **Deploying Edge AI on ESP32 Devices**. Available at: <https://docs.edgeimpulse.com/docs/edge-ai-hardware/mcu>
- [esp32_tflite] TensorFlow Lite for Microcontrollers. **Running TinyML Models on ESP32**. Available at: <https://www.tensorflow.org/lite/microcontrollers>
- [esp32_ai_projects] Jomjol / GitHub. **AI-on-the-Edge Device with ESP32**. Available at: <https://jomjol.github.io/AI-on-the-edge-device-docs/>
- [esp32_smartirrigation] Academia.edu. **Edge AI IoT Smart Irrigation with ESP32**. Available at: https://www.academia.edu/117576198/IoT_Based_Smart_Irrigation_System
- [timeseries_anomaly_arxiv] Arxiv.org. **Anomaly Detection in Time Series Data for IoT Systems**. Available at: <https://arxiv.org/abs/2006.05913>
- [tinymml_preprocessing_tflite] TensorFlow Lite for Microcontrollers. **Preprocessing and Feature Extraction for TinyML**. Available at: <https://www.tensorflow.org/lite/microcontrollers/overview>
- [esp32_sensor_data_github] GitHub. **ESP32 Sensor Data Collection and Preprocessing**

Examples. Available at: <https://github.com/EdgeML/ESP32-Sensor-Tutorial>

- [soil_moisture_mendeley] Mendeley Data. **Soil Moisture and Temperature Dataset**. Available at: <https://data.mendeley.com/datasets/fpbfmtc9vnm>
- [neon_soil_dataset] NEON. **Soil and Ecological Restoration Dataset**. Available at: <https://www.neonscience.org>
- [soil_moisture_longterm] ESSD. **10-Year Soil Moisture and Temperature Dataset**. Available at: <https://essd.copernicus.org/articles/14/5513/2022/>
- [growliflower_dataset] Arxiv. **GrowliFlower Plant Growth Dataset**. Available at: <https://arxiv.org/abs/2204.00294>
- [kaggle_biosensor_dataset] Kaggle. **Smart Plant Biosensor Monitoring Dataset**. Available at: <https://www.kaggle.com/datasets/muqaddasejaz/smart-plant-biosensor-monitoring-dataset>
- [kaggle_indoor_growth] Kaggle. **Indoor Plant Health and Growth Dataset**. Available at: <https://www.kaggle.com/datasets/souvikrana17/indoor-plant-health-and-growth-dataset>
- [kaggle_plant_health_data] Kaggle. **Plant Health Data**. Available at: <https://www.kaggle.com/datasets/ziya07/plant-health-data>
- [kaggle_plant_health_dataset] Kaggle. **Plant Health Dataset**. Available at: <https://www.kaggle.com/datasets/manutrex78/plant-healt-dataset>
- [kaggle_disease_dataset] Kaggle. **Plant Health Diseases Dataset**. Available at: <https://www.kaggle.com/datasets/muqaddasejaz/plant-health-diseases-dataset>
- [kaggle_pathogen_dataset] Kaggle. **Pathogen Dataset (Image-based Plant Disease Classification)**. Available at: <https://www.kaggle.com/datasets/kanishk3813/pathogen-dataset>
- [tinymml_book] Warden, P., & Situnayake, D. **TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers**. O'Reilly Media, 2019.
- [tflite_micro] TensorFlow. **TensorFlow Lite for Microcontrollers**. Available at: <https://www.tensorflow.org/lite/microcontrollers>
- [esp32_tflite_blog] TensorFlow Blog. **TensorFlow Lite Micro Support for ESP32**. Available at: <https://blog.tensorflow.org/2020/08/announcing-tensorflow-lite-micro-esp32.html>
- [tinymml_practices] Meta Intelligence. **TinyML in Practice: Engineering Methods and Performance Benchmarks**. Available at: <https://www.meta-intelligence.tech/en/paper-tinymml.html>
- [esp32_tinymml_analysis] Arxiv. **TinyML: Analysis of Xtensa LX6 Microprocessor for Neural Network Applications by ESP32 SoC**. Available at: <https://arxiv.org/abs/2106.10652>
- [tinymml_navigation] Arxiv. **TinyNav: End-to-End TinyML for Real-Time Autonomous Navigation on Microcontrollers**. Available at: <https://arxiv.org/abs/2603.11071>
- [esp32_tinymml_tutorial] TeachMeMicro. **TinyML with ESP32 Tutorial**. Available at: <https://www.teachmemicro.com/tinymml-with-esp32-tutorial/>
- [tinymml_workflow] Sigma Blog. **ESP32 TinyML Workflow and Deployment**. Available at: <https://blog.sigma.com/esp32-tinymml-benchmark-energy-for-edge-intelligence/>